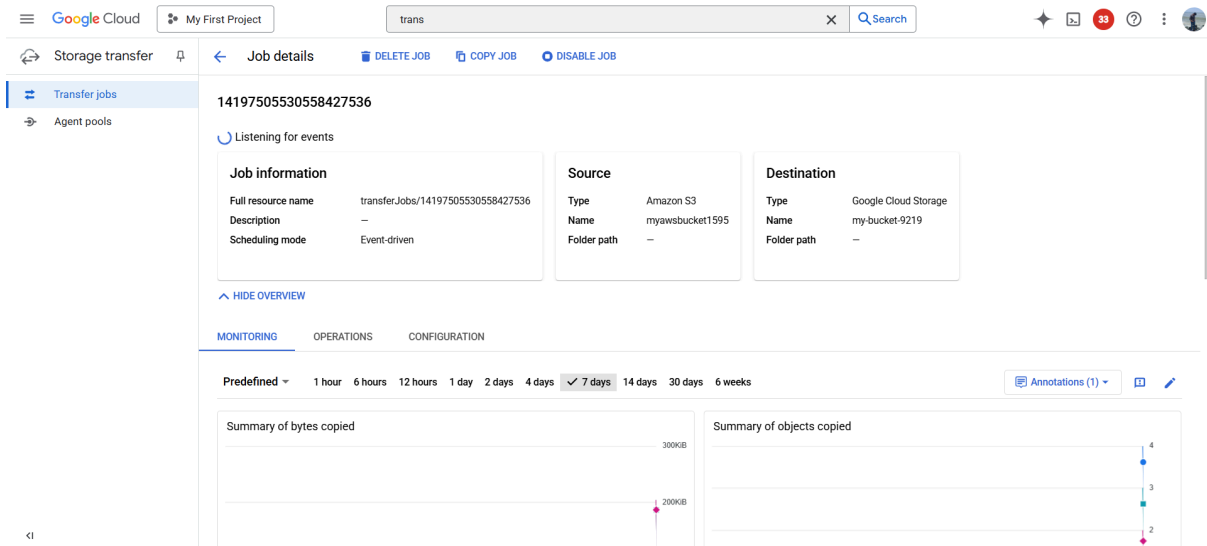


S3 to GCS & GCS to S3 File Transfer

1) To Transfer the Files from S3 to GCS. We will be using a Storage transfer service.

Step 1:

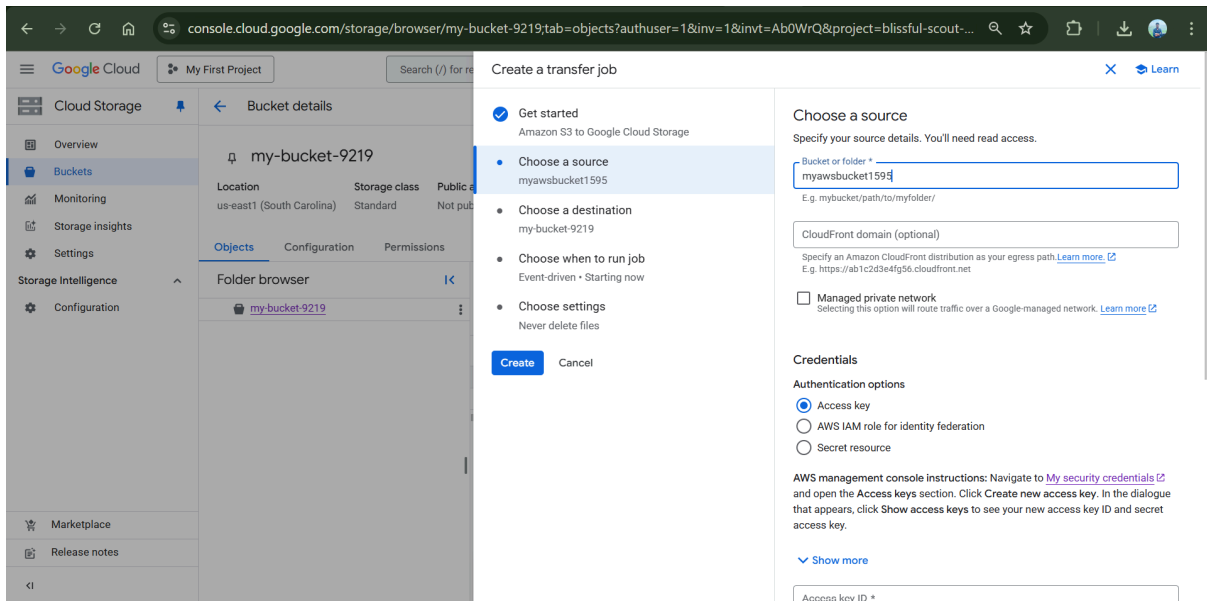
Click on Data Transfer in GCS. Use Transfer Job Only.



The screenshot shows the 'Create a transfer job' dialog. The 'Get started' section lists the steps: 'Get started', 'Choose a source', 'Choose a destination', 'Choose when to run job', and 'Choose settings'. The 'Get started' section also includes a 'Select S3-compatible' button. The 'Source type' is set to 'Amazon S3' and the 'Destination type' is set to 'Google Cloud Storage'. The 'Scheduling mode' is set to 'Event-driven' (marked as 'New'). The 'Looking for more options?' link is visible. The 'Create' button is highlighted.

Step 2:

Set up the source and Destination and the even driven stream name and access key everything what we need to specify.



Step 3:

For Access key. Create one user in IAM in AWS and specify the following permissions for S3 and SQS. Generate the keys and specify the keys in the Transfer service configuration.

IAM User Inline Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Access",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::my-bucket-1595",
        "arn:aws:s3:::my-bucket-1595/*"
      ]
    },
    {
      "Sid": "AllowSQSAccess",
      "Effect": "Allow",

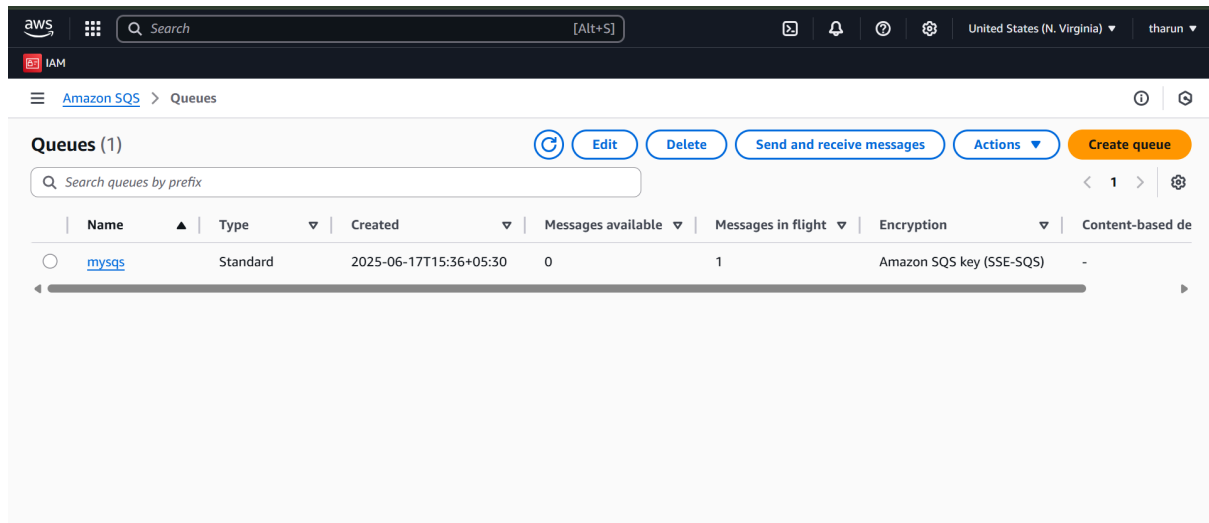
```

```

    "Action": [
      "sqs:ReceiveMessage",
      "sqs:GetQueueAttributes",
      "sqs:GetQueueUrl"
    ],
    "Resource": "arn:aws:sqs:eu-central-1:528757813453:my-sqs-1595"
  }
]
}

```

Step 4: Create SQS in AWS



Step 5:
While creating the SQS. defined the Access policy permissions according to who can access the queue.

SQS Policy Json:

```

{
  "Version": "2012-10-17",
  "Id": "example-ID",
  "Statement": [
    {
      "Sid": "example-statement-ID",
      "Effect": "Allow",

```

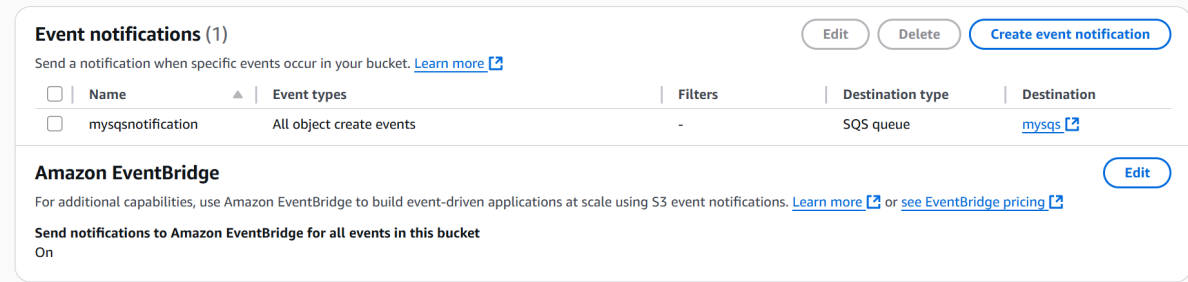
```

"Principal": {
  "Service": "s3.amazonaws.com"
},
"Action": "SQS:SendMessage",
"Resource": "arn:aws:sqs:us-east-1:905418149763:mysql",
"Condition": {
  "StringEquals": {
    "aws:SourceAccount": "905418149763"
  },
  "ArnLike": {
    "aws:SourceArn": "arn:aws:s3:::myawsbucket1595"
  }
}
}
]
}

```

Step 6:

After editing the policy. Will be adding the SQS in the Event stream in S3 Properties. And also enable the even-bridge notifications on.



The screenshot shows the 'Event notifications (1)' section in the AWS S3 console. It includes a table with one notification named 'mysqlnotification' that triggers on 'All object create events' and sends them to an 'SQS queue' named 'mysql'. Below the table, the 'Amazon EventBridge' section is visible, with a toggle switch set to 'On' for sending notifications to EventBridge for all events in the bucket.

☐	Name	Event types	Filters	Destination type	Destination
☐	mysqlnotification	All object create events	-	SQS queue	mysql

Amazon EventBridge

For additional capabilities, use Amazon EventBridge to build event-driven applications at scale using S3 event notifications. [Learn more](#) or [see EventBridge pricing](#)

Send notifications to Amazon EventBridge for all events in this bucket

On

Step 7:

After this click create in STS. The Transfer job will start and it's in progress. Whatever file we will be uploading in S3. same will transfer to GCS within seconds.

2) To Transfer the Files from GCS to S3. We will be using Datasync.

Step 1:
Create a Datasync task and specify the configuration.

The screenshot shows the AWS IAM console interface. The breadcrumb navigation at the top reads 'DataSync > Tasks > Create task'. On the left sidebar, 'Review' is selected under the 'Steps' section. The main content area is titled 'Configuration' and contains the following fields:

- Location type:** A dropdown menu with 'Object storage' selected.
- Region:** 'US East (N. Virginia) us-east-1'. A note below states: 'Transfers to or from Object storage must stay in the current Region.'
- Server:** A text input field containing 'storage.googleapis.com'. A label above it reads: 'Domain name or IP address of the object storage server.'
- Bucket name:** A text input field containing 'gcpbucket1595'. A label above it reads: 'Name of the bucket that will be used for the data transfer.'

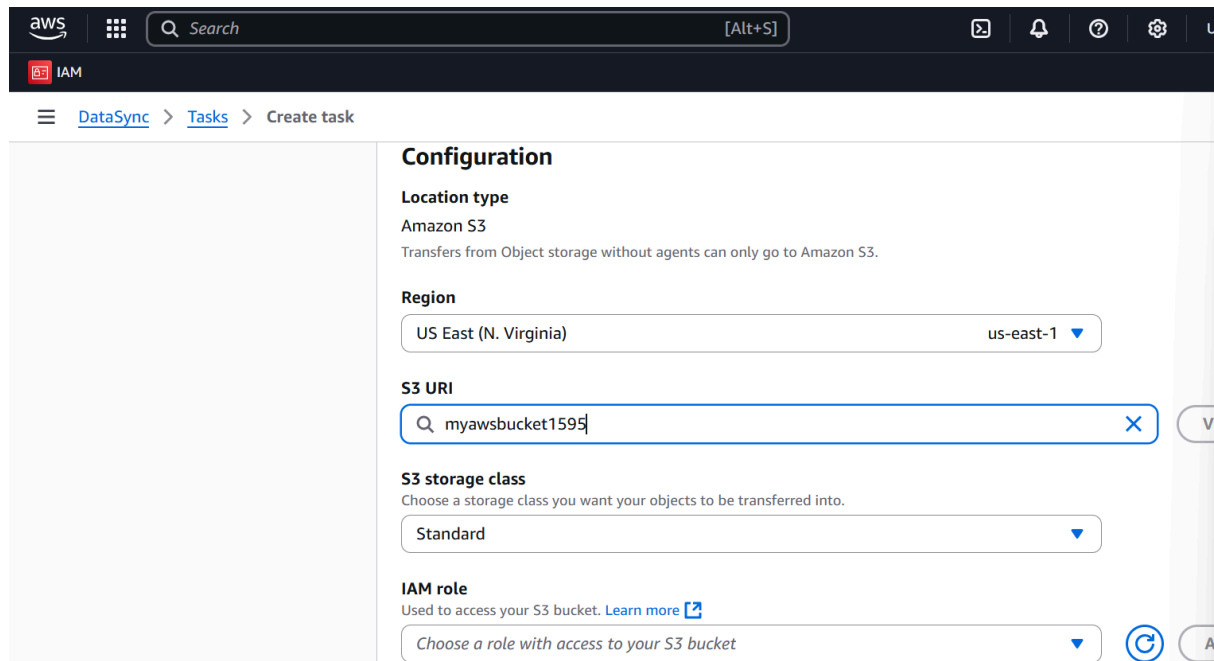
Step 2:
Specify the Accesskey and secret key from the GCP side to access the bucket.

The screenshot shows the Google Cloud console interface. The breadcrumb navigation at the top reads 'Cloud Storage > Settings'. The left sidebar shows 'Settings' selected under 'Cloud Storage'. The main content area is titled 'Settings' and contains the following sections:

- Access keys for service accounts:** A dropdown menu showing 'gcs-transfer-svc@blissful-scout-427512-n4.iam.gserviceaccount.com'. A button below reads '+ Create a key for another service account'.
- User account HMAC:** A section with a description: 'You can authenticate yourself when making requests to Cloud Storage using access keys tied to your user account instead of your organisation's service accounts. With this option, principals with access to your organisation maintain their own access keys and set their own default projects. Note that service account HMAC authentication is recommended for production workloads, to reduce administrative oversight and ensure continuity. [Learn more](#)'.
- Default project for interoperable access:** A section with a description: 'The Interoperability API uses your default project for all create bucket and list bucket requests made from your user account.' Below this is a message: 'You haven't set a default project for your user account yet' and a button: 'Set blissful-scout-427512-n4 as default project'.
- Access keys for your user account:** A section with a table containing two columns: 'Access key' and 'Secret'.

Step 3:

Specify the Destination configuration in Datasync.



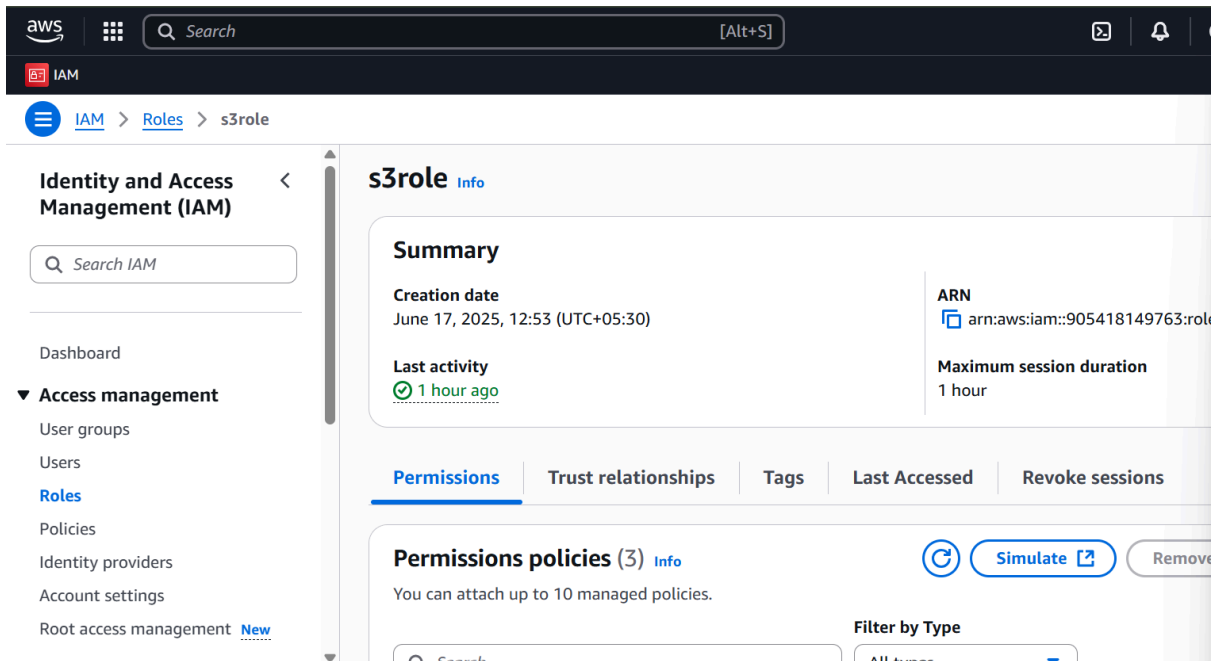
The screenshot shows the AWS DataSync console's 'Create task' configuration page. The breadcrumb navigation at the top indicates the path: DataSync > Tasks > Create task. The main configuration area is titled 'Configuration' and includes the following sections:

- Location type:** Set to 'Amazon S3'. A note below states: 'Transfers from Object storage without agents can only go to Amazon S3.'
- Region:** A dropdown menu showing 'US East (N. Virginia)' with 'us-east-1' selected.
- S3 URI:** A text input field containing 'myawsbucket1595'.
- S3 storage class:** A dropdown menu showing 'Standard'.
- IAM role:** A dropdown menu with the text 'Choose a role with access to your S3 bucket' and a 'Learn more' link.

On the right side of the configuration panel, there are buttons for 'V' (View) and 'A' (Add), along with a circular refresh icon.

Step 4:

And attach the IAM role to the Datasync with required permissions.

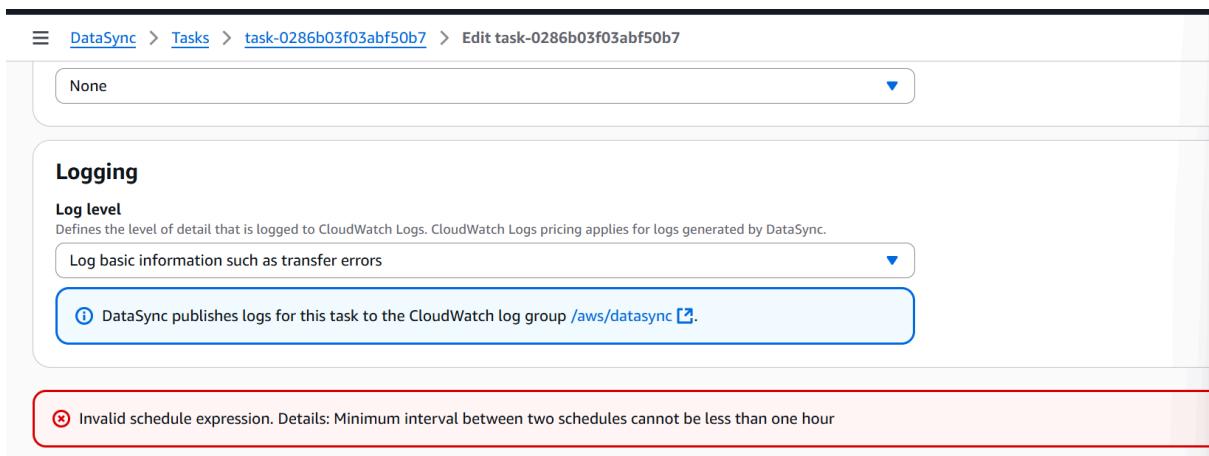


Step 5:

After setting up the datasync configuration. Try to upload one file in GCS Bucket. It will appear in the S3 Bucket. It will take some time.

Step 6:

To schedule the Datasync task. We will be using a schedule option in Datasync itself. But we can only schedule the cron job for 60 min. Not less than that.



Step 7:

To schedule the cron job in less than 60 min. We can use the Lambda function with the Event-bridge trigger. It will trigger the Datasync.

Datasync won't support the Schedule cron job less than 60 min. Minimum is 60 min.

Lambda function to trigger the Datasync for every 1 min or 5 min:

Step 1:

Create one lambda function with an inline code editor or upload the zip file.

Source Code:

```
import boto3

def lambda_handler(event, context):
    client = boto3.client('datasync')
    task_arn =
'arn:aws:datasync:us-east-1:905418149763:task/task-0286b03f03abf50b7'

    # Get all executions of the task
    executions =
client.list_task_executions(TaskArn=task_arn)['TaskExecutions']

    for execution in executions:
        exec_arn = execution['TaskExecutionArn']
        exec_desc =
client.describe_task_execution(TaskExecutionArn=exec_arn)
        status = exec_desc['Status']

        if status in ['QUEUED', 'LAUNCHING', 'PREPARING',
'TRANSFERRING', 'VERIFYING']:
            print(f"Execution already running with status: {status}")
            return

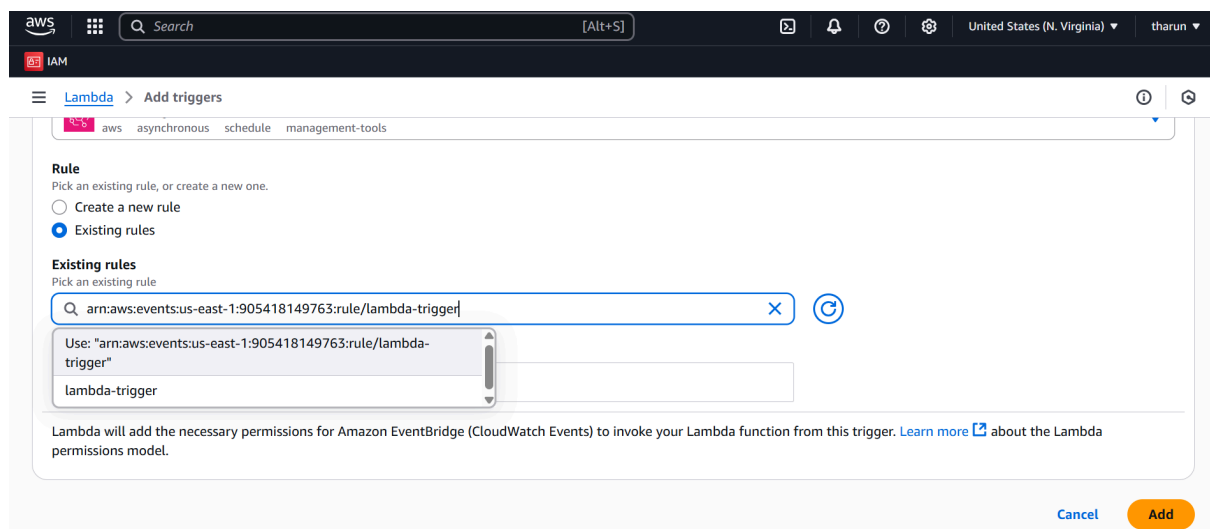
    # If no execution is currently running
    response = client.start_task_execution(TaskArn=task_arn)
    print("Started new DataSync execution:",
response['TaskExecutionArn'])
```


Step 2:

Give the required Datasync Permissions to Lambda function IAM Role for the Datasync.

Step 3:

We will be using Event-bridge as a trigger. To run the function for every 1 min or 5 min we will be scheduling the job.



The screenshot shows the AWS IAM console interface for adding triggers to a Lambda function. The breadcrumb navigation shows 'Lambda > Add triggers'. The page has tabs for 'aws', 'asynchronous', 'schedule', and 'management-tools', with 'aws' being the active tab. Under the 'Rule' section, there are two radio buttons: 'Create a new rule' (unselected) and 'Existing rules' (selected). Below this, the 'Existing rules' section prompts the user to 'Pick an existing rule'. A search bar contains the text 'arn:aws:events:us-east-1:905418149763:rule/lambda-trigger'. A dropdown menu is open, showing two options: 'Use: "arn:aws:events:us-east-1:905418149763:rule/lambda-trigger"' and 'lambda-trigger'. At the bottom of the form, there is a note: 'Lambda will add the necessary permissions for Amazon EventBridge (CloudWatch Events) to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.' The bottom right corner features a 'Cancel' button and an 'Add' button.

